

Debugging in Emacs: The Grand Unified Debugger



This article briefly explores the features of the Grand Unified Debugger, a debugging tool for Emacs.

If you are a C/C++ developer, it is highly likely that you have crossed paths with GDB (the GNU debugger) which is, without doubt, one of the most powerful and unrivalled debuggers out there. Its only drawback is that it is command line based, and though that offers a lot of power, it is sometimes a bit restrictive as well. This is why smart people started coming up with IDEs to integrate editors and debuggers, and give them a GUI. There are still developers who believe that using the mouse reduces productivity and that mouse-click based GUIs are temptations by the devil.

Since Emacs is one of the coolest text editors out there, I am going to show you how to write, compile and debug code without having to touch the mouse or move out of Emacs.

The Grand Unified Debugger, or GUD as it is commonly known, is an Emacs mode in which GDB can be run from within Emacs. This provides all the features of Emacs in GDB. The user does not have to move out of the editor to debug the code written.

Setting the stage for the Grand Unified Debugger

If you are using a Linux machine, then it is likely you will have GDB and gcc already installed. The next step is to ensure that Emacs is also installed. I am assuming that the readers are familiar with GDB and have used it at least for basic debugging. If not, please do check out some quick introductions to GDB

that are widely available on the Internet.

For people who are new to Emacs, let me introduce you to some basic terminology. Throughout this article, you will see shortcut commands such as *C-c*, *M-x*, etc. *C* means the *Ctrl* key and *M* means the *Alt* key. *C-c* means the *Ctrl + c* keys are pressed. If you see *C-c c*, it means *Ctrl + c* is pressed followed by *c*. Also, in Emacs, the main

The screenshot shows the Emacs editor interface. The main window displays a C++ program: `1 #include<iostream>`, `2 #include <assert.h>`, `3`, `4 int factorial(int num) {`, `5 int product = 1;`, `6 while(num--){`, `7 product *= num;`, `8 }`, `9 return product;`, `10 }`, `11`, `12 int main() {`, `13 int result = factorial(5);`, `14 assert(result == 120);`, `15 return 0;`, `16 }`, `17`, `18`. The mini-buffer at the bottom shows the command line: `- 233 -: buggyFactorial.cpp C++/l company FlyC Abbrev 16 : 1 All` and the compile command: `Compile command: g++ buggyFactorial.cpp -g`.

Figure 1: Compile command in Emacs' mini buffer